Rust своими руками: автономное хранилище рыночных данных

Практический учебник проектирования и миграции market-storage из Python в Rust

Один бинарный файл · существующий бот · тот же VPS · без PyO3 и maturin

Издание 1.1 · 13 ноября 2026 года

Как работать с книгой

Эта книга построена по образцу русских книг Rust из каталогов `recomendations/` и `backtest/`: объяснение предметной задачи, постепенное освоение Rust, самостоятельные упражнения и проверяемая приёмка. Учебный проект здесь другой: `paper/server_bot/storage`. Вы создаёте самостоятельное приложение для сохранения, чтения и передачи рыночных наблюдений, а не вычислительное ядро торговой стратегии.

Предполагается, что читатель понимает Python-функции, классы, исключения и работу с файлами. Предварительно знать Rust не обязательно. Книга объясняет необходимые понятия на задачах хранилища; для синтаксических подробностей полезно параллельно читать The Rust Programming Language.

По условию этого проекта книга не содержит программной реализации, конфигурационных листингов или готового Rust-кода. Это учебник проектирования: он даёт контракты, последовательности действий, таблицы решений, задачи и критерии готовности. Python-модули дополнены проверяемым накопительным merge и квитанциями копирования; Rust-реализации в этом каталоге пока нет. Описанные Rust-команды, протоколы и гарантии являются предложением, пока их не подтвердит реализация и тесты.

Каждую главу проходите в три шага. Сначала объясните правило своими словами. Затем составьте тест с конкретным входом и ожидаемым результатом. После самостоятельной реализации проверьте результат и запишите причину принятого решения. Не переходите к следующему этапу только потому, что программа собралась.

Задания A проверяют понимание, B требуют реализации в отдельном учебном проекте, C проверяют поведение при отказах. В книге 24 главы и 72 задания H01–H72. Завершающий маршрут M01–M10 соединяет их в производственную миграцию.

Содержание

1. Что именно мы переписываем
2. Как данные проходят через существующую систему
3. Бинарный файл на том же VPS
4. Первые понятия Rust на языке хранилища
5. Предметные типы и проверка входов
6. Владение, заимствование и ограниченная память
7. Ошибки и наблюдаемый результат
8. Время и причинность
9. SQLite и совместимость схемы
10. Запись и граница подтверждения
11. Сжатие и адресация по содержимому
12. Чтение архива и пакеты GEX
13. Расписание и повторные попытки
14. Retention и освобождение места
15. Согласованный online backup
16. SSH и загрузка снимков
17. Атомарная публикация и gzip
18. CLI как публичный интерфейс
19. Подключение существующего Python-бота
20. Тестирование и доказательство совместимости
21. Производительность и выбор библиотек
22. Сборка и установка на VPS
23. Эксплуатация, диагностика и восстановление
24. Миграция, rollback и итоговая приёмка

Приложения: матрица контрактов, словарь, источники и границы проверенности.

Дополнение издания 1.1: проверенная история, квитанции и удаление через 48 часов; задачи R01–R07.

1. Что именно мы переписываем

Ответственность пакета

Исходный пакет `market_storage` хранит точные текстовые тела ответов бирж вместе с метаданными запроса. Он поддерживает обычные и сжатые записи SQLite, чтение по времени получения, удаление старых наблюдений, согласованные снимки и скачивание по SSH. В производственном Python-пакете нет сторонних зависимостей: используются стандартные SQLite, zlib, gzip, asyncio и файловые операции.

Это не сборщик котировок целиком. HTTP-запросы и проверка ответов бирж находятся в `paper/server_bot/server_bot/server_bot/tunnel_archive.py`. Вычисления GEX и осцилляторов находятся вне storage. Переписать только каталог storage — не значит перенести весь pipeline на Rust.

Цель первого выпуска: Rust владеет операциями хранения и переноса, Python продолжает собирать данные и вычислять индикаторы. Новый процесс запускается отдельно. Python передаёт ему сериализованные запросы, а не вызывает Rust через импорт.

Карта исходников

Исходный файл	Ответственность	Будущий Rust-модуль
`models.py`	Активы, источники, наблюдение, окно	domain
`schema.py`	Создание таблиц и совместимое расширение	schema
`archive.py`	Запись, чтение, due, prune	archive
`inputs.py`	Комплектные GEX-наборы	inputs
`backup.py`	Online backup и проверка	backup
`transfer.py`	SSH и безопасная публикация	transfer
`download.py`	Накопительный архив и gzip	download
`merge.py`	Проверка, дедупликация и timeline	merge
`receipts.py`	Подтверждение копирования и очередь удаления	receipts
`writing.py`	Запись внутри транзакции владельца	writing
`__main__.py`	Export и pull	cli

Модули — границы ответственности внутри одного Rust-пакета, не обязательные отдельные crates. На старте достаточно библиотеки с логикой и одного исполняемого приложения.

Практикум H01–H03

H01 · A. Нарисуйте границу storage. Приёмка: HTTP и формулы индикаторов находятся снаружи.

H02 · B. Составьте перечень публичных операций Python. Приёмка: учтены store, load, due, prune, load_inputs, snapshot, validate, pull и dated download.

H03 · C. Объясните, почему замена только export не завершает миграцию. Приёмка: указаны прямые импорты в сборщике и калькуляторе.

2. Как данные проходят через существующую систему

От ответа биржи до расчёта

Сборщик запоминает время начала запроса, получает ответ, сохраняет его текст, проверяет HTTP/API-результат и создаёт Observation. Время available фиксируется при подготовке записи после обработки ответа. Затем вызывается Archive.store. Следовательно, available — записанная граница получения и обработки, а не точный момент COMMIT. Запись становится доступной читателю только после фиксации SQLite.

Ошибка запроса тоже является свидетельством: сохраняются доступное тело и классификация ошибки. Читатель расчётных входов исключает такие строки. Нельзя выбрасывать неуспешные ответы только потому, что они не используются индикатором.

Калькулятор архива в `backtest/gui_calculator/archived_market.py` импортирует Archive и load_inputs напрямую. Он читает уже сохранённые наблюдения; этот путь не подменяет отсутствующие данные живым HTTP-запросом.

Два интерфейса вместо одного

Сейчас пакет одновременно предоставляет Python API и CLI. Будущий бинарный файл естественно заменяет CLI, но Python API автоматически не исчезает. Требуется небольшой Python-адаптер процесса либо явно согласованный переход потребителей к новому интерфейсу.

В этой книге конечная цель — Rust как единственная реализация storage-правил, Python как транспортный адаптер и потребитель. Не следует оставлять две независимые реализации группировки GEX, расписания и удаления записей.

Практикум H04–H06

H04 · A. Проследите один успешный и один неуспешный запрос до базы.

H05 · B. Составьте таблицу потребителей и требуемых операций. Приёмка: отдельно перечислены сборщик, расчёт архива и downloader.

H06 · C. Опишите отказ записи после успешного HTTP. Приёмка: сборщик не считает наблюдение надёжно сохранённым и не начинает зависимый расчёт на неподтверждённых данных.

3. Бинарный файл на том же VPS

Выбор первого интерфейса

Для первого выпуска выбираем CLI с конечными операциями. Бот запускает процесс для запроса due, записи наблюдения или чтения окна. Export запускается по SSH на VPS, pull — на локальной машине. PyO3, maturin, Python wheels и общий адресный простор не нужны.

Процесс store читает одно наблюдение из stdin и завершается после COMMIT. Процесс load выдаёт поток наблюдений. Это проще постоянного сервиса и сохраняет существующую границу отдельной записи. Стоимость запуска следует измерить: она не считается заранее ни критичной, ни бесплатной.

Если измерения покажут существенные накладные расходы, следующим вариантом станет длительно живущий дочерний процесс с построчным протоколом. Unix socket нужен только при независимом жизненном цикле и нескольких клиентах. HTTP-сервис на первом этапе не даёт необходимого преимущества.

Совместное размещение

Бот и бинарный файл работают на одной машине и обращаются к одному локальному архиву. Локальное взаимодействие не требует SSH. Бинарному файлу нужны права на каталог базы, журналы SQLite и временные файлы. Лучше запускать его от того же сервисного пользователя, что и бота, без root.

Systemd продолжает управлять ботом. Для локальных backup-задач можно использовать отдельный timer. Не следует создавать timer, независимо дублирующий сбор данных существующего бота.

Практикум H07–H09

H07 · A. Разделите локальные и удалённые процессы на схеме.

H08 · B. Опишите жизненный цикл одного вызова store: запуск, stdin, COMMIT, ответ, завершение.

H09 · C. Спроектируйте обработку отсутствующего бинарного файла. Приёмка: бот сообщает операционную ошибку, а не продолжает работу с фиктивным успехом.

4. Первые понятия Rust на языке хранилища

Что изучать сначала

Cargo управляет сборкой и зависимостями. Package содержит библиотечную часть и binary target. Библиотека позволяет тестировать storage-операции без запуска оболочки; бинарная часть разбирает аргументы и вызывает эти операции.

Struct описывает запись с именованными полями. Enum ограничивает набор допустимых источников или режимов. Result выражает успех либо ошибку. Option выражает отсутствие значения: отсутствие необязательного collection_started не является отказом.

Pattern matching полезен для явного выбора raw или compact. Он помогает увидеть все варианты. Trait нужен при реальной общей границе, например при подмене часов; не требуется превращать каждую функцию в универсальный framework.

Rust проверяет типы и время жизни ссылок, но не знает, что available не должно предшествовать requested. Эти правила вы задаёте конструкторами и тестами.

Порядок обучения

Сначала создайте пустую библиотеку и CLI. Затем реализуйте проверяемый Asset, затем временное окно, затем один тест ошибки. Только после этого подключайте SQLite. Так ошибки сборки, модели и файловой системы не смешиваются в одну задачу.

Практикум H10–H12

H10 · A. Объясните отличие ошибки от отсутствующего значения на примере архива.

H11 · B. Подготовьте учебный package с отдельными domain и cli. Приёмка: доменные тесты не запускают subprocess.

H12 · C. Докажите, что успешная компиляция не предотвращает обратное временное окно.

5. Предметные типы и проверка входов

Нельзя доверять сериализации

JSON из собственного Python-бота всё равно является внешним входом для Rust. Проверяются структура, типы, обязательные поля, ограничения размера и предметные отношения. Наличие Deserialize само по себе не доказывает корректность Observation.

Asset допускает BTC и ETH. Source допускает девять существующих значений. Mode допускает raw и compact. Неизвестное имя нового источника не должно незаметно становиться произвольной строкой в новой записи.

Observation содержит asset, source, requested, available, payload, collection и error. Collection непустая, временные значения содержат UTC offset, available не раньше requested. Payload сохраняется текстом, даже если это некорректный JSON биржи. Проверять успешность конкретного API должен сборщик.

Window включает asset, start и end. Start не позже end. Обе границы включительные. Для конфигурации транспорта и retention используйте отдельные типы: адрес SSH не является настройкой политики удаления.

Совместимость и усиление правил

Не добавляйте новое запрещающее ограничение без анализа существующих данных. Например, сегодня payload может быть пустым у неуспешного запроса. Новый валидатор обязан это допускать. Усиление проверок повреждённой базы допустимо, но его надо описать как изменение поведения.

Практикум H13–H15

H13 · A. Перечислите инварианты Observation и Window.

H14 · B. Разделите декодирование JSON и создание проверенного доменного значения.

H15 · C. Проверьте неизвестный source, пустую collection, неверный тип payload и naive timestamp. Приёмка: база не изменена.

6. Владение, заимствование и ограниченная память

Кто владеет ответом

Owned String подходит для сохранённого тела ответа. Заимствованная строка подходит для временного чтения внутри операции, пока владелец существует. Передача значения означает передачу ответственности за его срок жизни, а не обязательное копирование всех байтов.

Не превращайте каждую строку в новый String при прохождении модулей. Но и не усложняйте первый проект системой ссылок, живущих дольше SQLite statement. Декодированная Observation с собственными значениями — понятная граница между SQL и доменной логикой.

Потоковая обработка

Backup, SSH и gzip обрабатываются ограниченными буферами. Размер архива не должен определять размер оперативной памяти downloader. Для чтения отдельных payload задаётся лимит распаковки: небольшой сжатый BLOB не гарантирует маленький результат.

Крупный load может быть построчным, но формирование GEX-пакетов требует состояния по collection. Для первой версии ограничьте размер окна и явно измерьте память. Потоковый вывод не делает всю операцию автоматически O(1) по памяти.

Практикум H16–H18

H16 · A. Укажите владельца строки payload на каждом этапе.

H17 · B. Спроектируйте export без чтения целого файла в память.

H18 · C. Проверьте превышение лимита распаковки. Приёмка: понятная ошибка и ограниченный расход памяти.

7. Ошибки и наблюдаемый результат

Предметная ошибка и операционный отказ

Неверное окно, занятая база, повреждённый zlib, ошибка SSH и заполненный диск имеют разные причины. Сохраняйте структурированную классификацию внутри приложения, а на границе CLI возвращайте устойчивый код и безопасное описание.

Предлагаемый контракт: 0 — завершённый успех, 2 — неверный ввод или несовместимый протокол, 1 — операционная ошибка. Более подробные категории можно передавать в JSON-диагностике stderr. Эта схема новая и не описывает уже существующий Rust CLI.

Stdout является каналом результата. Для export это исключительно бинарные байты SQLite. Ни строка приветствия, ни progress bar туда не допускаются. Диагностика идёт в stderr. Panic не является штатным способом сообщить пользователю о неверной дате.

Неоднозначный результат

Процесс мог выполнить COMMIT и погибнуть до выдачи подтверждения. Родитель знает только, что подтверждение отсутствует. Сегодня store не имеет ключа идемпотентности и допускает повторные строки. Поэтому обещать exactly-once поверх subprocess нельзя.

Store сохраняет эту семантику и явно сообщает неопределённый исход. Merge скачанных снимков отдельно удаляет точные повторы по временному ключу; это не делает store идемпотентным. Автоматический повтор store допустим только после решения о дубликатах. Для будущего idempotency key нужна отдельная миграция схемы и контракт уникальности; content digest не заменяет идентификатор наблюдения.

Практикум H19–H21

H19 · A. Разделите ошибки пользователя, транспорта и данных.

H20 · B. Опишите stdout/stderr и exit status для каждой операции.

H21 · C. Смоделируйте COMMIT без ответа. Приёмка: система не объявляет доказанное отсутствие записи.

8. Время и причинность

Три разных времени

Requested — начало запроса. Available — сохранённая граница получения и обработки. Collection minute — начало минуты коллекции, вычисленное из collection_started либо requested. Эти поля нельзя заменить одним timestamp.

Расчёт на cutoff T не видит наблюдение с available позже T, даже если запрос начался раньше. Граница start и cutoff end включительны. Ошибки и неизвестные legacy-источники не становятся расчётными входами.

Формат SQLite — часть контракта

Python нормализует время в UTC и сохраняет ISO-строку с `+00:00`. Нулевые микросекунды обычно отсутствуют, ненулевые представлены шестью цифрами. SQL сравнивает TEXT, поэтому другой вывод, например Z или иное число дробных знаков, нельзя вводить без теста на границах.

Для совместимого первого выпуска Rust должен воспроизводить Python-формат и точность до микросекунд. В новом stdin-протоколе разумно принимать offset-aware время с точностью не выше микросекунд; более точные значения отвергать, чтобы не вводить скрытое округление.

Смешанные внешние legacy-форматы требуют отдельного аудита. Если необходимо исправить хранение на целочисленное время, это новая версия схемы с планом преобразования, а не побочный эффект переписывания.

Практикум H22–H24

H22 · A. Разберите запрос до cutoff с ответом после cutoff.

H23 · B. Подготовьте пары Python/Rust для нулевых и ненулевых микросекунд, разных offsets и границы суток.

H24 · C. Проверьте наблюдения точно в start/end и на микросекунду снаружи. Приёмка: совпадают SQL-выборки обеих реализаций.

9. SQLite и совместимость схемы

Три таблицы

`tunnel_inputs` содержит raw payload и метаданные. `tunnel_payloads` содержит digest, codec и сжатый BLOB. `tunnel_observations` содержит compact-метаданные со ссылкой на digest. Индексы организованы по asset, source и available_at.

Существующий prepare создаёт таблицы и добавляет отсутствующие collection_id и minute_at к raw-таблице. Старые строки могут иметь NULL в новых полях. Читатель назначает им отдельную synthetic identity вида legacy:raw:id; он не реконструирует коллекцию по соседним временам.

Правила первого выпуска

Сохраните имена, типы колонок и смысл данных. Не объединяйте raw и compact автоматически. Оба представления могут содержать наблюдения, и текущий читатель учитывает оба. SQL foreign key declaration не означает, что каждая историческая connection реально включала enforcement.

Для новых versioned migrations нужен явный механизм: проверка структуры и версия схемы. Старые базы без версии сначала распознаются по структуре. Не помечайте повреждённую или неизвестную базу «последней версией» только после CREATE IF NOT EXISTS.

Read-only операции не создают отсутствующий файл. Для due отсутствующий архив означает, что источник пора запросить; для load и export это ошибка.

Практикум H25–H27

H25 · A. Объясните разницу digest payload и идентификатора observation.

H26 · B. Подготовьте fixtures старой raw, compact и смешанной базы.

H27 · C. Проверьте read-only путь к отсутствующему файлу. Приёмка: файл и каталог не созданы чтением.

10. Запись и граница подтверждения

Одна запись — одна фиксация

Сегодня каждый ответ сохраняется независимо. Полный набор GEX не является одной общей транзакцией. Это позволяет сохранить часть коллекции при падении сборщика. Rust должен удержать эту семантику на первом этапе.

Raw-запись вставляет строку. Compact-запись вычисляет digest, добавляет отсутствующий payload и вставляет observation в одной транзакции. Ошибка между операциями не должна оставлять подтверждённое наблюдение без тела.

Настройки надёжности

Текущий код открывает SQLite с timeout 30 секунд; гарантии синхронизации зависят также от SQLite и файловой системы. В целевой системе задайте busy timeout и документируйте journal_mode/synchronous. WAL не следует объявлять включённым по умолчанию только потому, что тесты умеют проверять его.

Положительный ответ выдаётся после успешного COMMIT. При завершении с ошибкой родитель не подменяет результат локальным кэшем. Закрытие stdin и корректная обработка stderr обязательны, иначе родитель может зависнуть на трубах.

Практикум H28–H30

H28 · A. Опишите транзакцию compact store.

H29 · B. Проверьте запись Python → чтение Rust и обратное направление.

H30 · C. Внесите отказ между payload и observation. Приёмка: rollback сохраняет согласованность.

11. Сжатие и адресация по содержимому

Хэшируем исходный текст

Digest — SHA-256 от UTF-8 байтов payload. Нельзя сначала разобрать JSON и сериализовать его заново: порядок полей, пробелы и переносы являются частью сохранённого свидетельства.

Codec текущей compact-записи — zlib. Это не gzip и не произвольный raw DEFLATE. Rust-библиотека должна читать Python zlib и создавать данные, которые Python может распаковать. Совпадение сжатых байтов не требуется; требуется совпадение распакованных UTF-8 байтов и digest.

Дедупликация разделяет одинаковый payload между разными observations. Время, source и collection остаются у каждого наблюдения. Одинаковое тело не означает, что два события получения можно объединить.

Проверка целостности

Текущий decode распаковывает тело, но не сверяет digest и не выбирает decoder по codec. В Rust рекомендуется добавить явную проверку codec и digest. Это усиление обнаружения повреждений, а не доказанная существующая гарантия Python-пакета. Нормальные файлы остаются совместимыми, повреждённые могут отклоняться раньше.

Практикум H31–H33

H31 · A. Объясните, почему одинаковый JSON с другими пробелами даёт другой digest.

H32 · B. Подготовьте Unicode, пустую строку, многострочное тело и одинаковые тела разных коллекций.

H33 · C. Повредите BLOB и digest независимо. Приёмка: ошибка не превращается в пустой успешный ответ.

12. Чтение архива и пакеты GEX

Что возвращает load

Читатель выбирает успешные строки подходящего asset в receipt-time окне. Неизвестные legacy source, включая OHLCV и authenticated margin, пропускаются. Raw и compact объединяются и сортируются по available. Ревизии сохраняются как отдельные observations.

Внутри каждой таблицы используется порядок available_at, id. Между таблицами при равных available текущая реализация зависит от порядка обнаружения таблиц и стабильной сортировки. Это не надёжно заданный межтабличный контракт. Для Rust явно определите tie-breaker и согласуйте его с потребителями, не выдавая новый порядок за гарантированно прежний.

Правило GEX

Один пакет содержит instruments, options и spot из одной collection. Время пакета — максимум available трёх частей. Неполные группы пропускаются. Два успешных наблюдения одного GEX source внутри одной collection вызывают ошибку; нельзя молча выбрать последнее.

Другие источники возвращаются отдельными потоками. Индивидуальная структура биржевого JSON проверяется расчётным адаптером. Storage гарантирует происхождение и временные границы, а не финансовую корректность полей ответа.

Практикум H34–H36

H34 · A. Составьте таблицу: полный набор, неполный, ошибочный source, duplicate source.

H35 · B. Подготовьте fixture двух соседних collection с отсутствующей частью. Приёмка: части не смешиваются.

H36 · C. Создайте одинаковые available в raw и compact. Приёмка: порядок явно описан и проверен.

13. Расписание и повторные попытки

Due — наблюдение состояния

Шесть market-источников используют период 300 секунд; три Bitget lending-источника — 3600 секунд. Начало bucket вычисляется по UTC epoch. Источник не due, если существует успешная запись с requested не раньше начала bucket и available не позже now.

Ошибка не закрывает bucket. После рестарта повторное чтение базы сохраняет расписание. Но due сам по себе не резервирует работу: два процесса могут одновременно увидеть True.

Где находится блокировка

Текущий сборщик удерживает file lock от проверки due через HTTP и запись до prune. Это защищает последовательность, а не только отдельный INSERT. В первом Rust-выпуске Python продолжает владеть этой блокировкой, а вызываемые процессы не захватывают её повторно: вложенный захват может заблокировать родителя.

Если позже сборщик переносится в Rust, владение lock переходит целиком. Все конкурирующие collectors должны использовать один согласованный путь. SQLite write lock не заменяет lock полного цикла планирования.

Практикум H37–H39

H37 · A. Рассчитайте bucket непосредственно до и после часовой границы.

H38 · B. Проверьте restart, успешный запрос и ошибку в одном bucket.

H39 · C. Запустите конкурирующие collectors. Приёмка: проверяется общий цикл под lock, а не только отсутствие повреждения SQLite.

14. Retention и освобождение места

Удаление по available

Удаляются наблюдения с available строго раньше cutoff. Строка точно на cutoff сохраняется. После удаления compact-observations удаляются payloads, на которые больше нет ссылок. Эти действия выполняются одной транзакцией.

Retention должен быть положительным. До включения copy-managed режима действует прежняя политика collector по возрасту, по умолчанию 24 часа. После создания copy_receipts prune удаляет только подтверждённые записи через 48 часов после первого ACK; возраст наблюдения больше не разрешает удалять нескопированное. Переход требует обновления storage на VPS и перезапуска collector.

Удаление строк обычно освобождает страницы для повторного использования, но не обязано уменьшать файл. VACUUM — отдельная операция с расходом времени, блокировок и свободного места. Не запускайте её после каждого цикла.

Опасная ошибка конфигурации

Если перепутать часы и секунды, база может потерять почти всю историю. Конфигурация должна содержать явную единицу измерения. Перед изменением production retention сравните её с максимальным интервалом скачивания и возможным простоем клиента.

Практикум H40–H42

H40 · A. Разберите три записи: до cutoff, на cutoff, после cutoff.

H41 · B. Проверьте общий payload старого и нового observations. Приёмка: payload сохраняется до удаления последней ссылки.

H42 · C. Смоделируйте остановку клиента дольше retention. Приёмка: документирована невозможность восстановить удалённый интервал из следующего снимка.

15. Согласованный online backup

Почему нельзя просто скопировать файл

Работающая SQLite может хранить зафиксированные изменения в WAL. Копия только основного файла не обязана содержать эти данные. Текущая реализация использует SQLite online backup API; Rust должен использовать соответствующую возможность выбранного binding.

Source открывается read-only. Destination — отдельный временный файл. Одинаковые source и destination запрещены. Для усиления защиты стоит учитывать существующие файловые alias, а не только совпадение текстовых путей.

Экспорт

Сначала создаётся завершённый backup, затем он проверяется и только потом потоково отправляется в stdout. Это удерживает большие файлы вне памяти, но требует временного дискового места на VPS.

Предлагаемая более строгая проверка включает quick_check, полную ожидаемую структуру, ссылки payload, поддерживаемый codec и при необходимости digest. Quick_check не доказывает полноту рыночной истории. Проверка всех payload может быть дорогой; разделите обязательную структурную проверку и явную глубокую проверку, сохранив честные названия гарантий.

Практикум H43–H45

H43 · A. Объясните роль WAL в backup.

H44 · B. Во время записей создайте backup и откройте его независимым соединением.

H45 · C. Проверьте заполнение временного диска и остановку export. Приёмка: ошибка не объявляется полным снимком.

16. SSH и загрузка снимков

Сохраняем системный OpenSSH

В первом выпуске Rust запускает установленный ssh. Это сохраняет SSH aliases, known_hosts, agent и пользовательские ключи. Переход на собственную SSH-библиотеку меняет поведение аутентификации и не нужен для цели самостоятельного binary.

Текущий транспорт использует BatchMode, ConnectTimeout 15 секунд и общий timeout 300 секунд. Эти значения можно сделать настройками, сохранив разумные defaults. При timeout дочерний процесс необходимо остановить и дождаться его завершения.

Две границы аргументов

Локальный ssh запускается списком аргументов без локальной shell-интерполяции. Но remote command обычно разбирается удалённой оболочкой. Следовательно, отдельно проверяются host, абсолютный remote root и относительный database path; отдельно обеспечивается POSIX quoting каждого удалённого аргумента.

Сейчас Remote.command запускает удалённый Python через uv. Миграция должна заменить эту команду на абсолютный путь установленного Rust export. Нельзя считать наличие локального Rust downloader доказательством независимости удалённой стороны от Python.

Практикум H46–H48

H46 · A. Разделите локальный argv и удалённую командную строку.

H47 · B. Проверьте пробелы, кавычки и shell-символы в допустимом root без выполнения посторонних действий.

H48 · C. Проверьте host с начальным дефисом, database с выходом через '..', отказ SSH и timeout. Приёмка: destination не изменён.

17. Атомарная публикация и gzip

Последовательность pull

Создайте staging в каталоге destination. Передайте stdout SSH непосредственно в файл. После успешного транспорта выполните flush и fsync, проверьте снимок, захватите локальную блокировку и постройте проверенное объединение со старой историей в отдельной базе. Только затем публикуйте объединение атомарным rename.

Staging должен находиться на той же файловой системе: перенос между файловыми системами не даёт того же свойства rename. Python merge синхронизирует готовый файл и каталог после rename перед отправкой ACK. В Rust необходимо сохранить этот порядок и отдельно проверить поддержку fsync на целевой файловой системе.

Destination следует считать снимком, который не используется как активно записываемая база. Замена имени при открытых readers оставляет старые handles на прежнем файле, а sidecar-файлы SQLite требуют отдельной осторожности. Новые расчёты открывают снимок после публикации.

Накопительные загрузки

Текущий gex-pull объединяет данные в tunnel_inputs.sqlite и обновляет gzip-копию. Если gzip не создан, проверенный SQLite остаётся, а команда завершается с ошибкой. ACK относится к SQLite, поэтому сбой gzip не отменяет подтверждённое копирование. Старые датированные файлы не удаляются и автоматически не импортируются.

Постоянное имя требует сериализации merge через отдельный lock-файл. Нельзя удалять lock-файл после разблокировки: ожидающий процесс мог уже открыть прежний inode. Gzip сравнивается после распаковки.

Практикум H49–H51

H49 · A. Объясните разницу atomic visibility и устойчивости к потере питания.

H50 · B. Проверьте замену старого снимка только после успешной validation.

H51 · C. Прервите transfer, validation и gzip по отдельности. Приёмка: предыдущее destination либо новый проверенный SQLite сохраняются согласно этапу.

18. CLI как публичный интерфейс

Предлагаемые операции

Команда	Вход	Выход	Изменяет архив
store	Observation через stdin, путь, режим	Подтверждение после COMMIT	Да
load	Asset, start, end, путь	Поток observations	Нет
inputs	То же окно	Наблюдения и GEX membership	Нет
due	Asset, source, now, путь	Boolean	Нет
prune	Now, retention, путь	Итог операции	Да
validate	Путь и уровень проверки	Результат проверки	Нет
export	Путь	Бинарный SQLite в stdout	Нет
pull	SSH-настройки, output	Результат публикации	Только destination
download	SSH-настройки, directory	Пути SQLite и gzip	Только local downloads
version	Нет	Версия binary и протокола	Нет

Это проект интерфейса. В существующем Python CLI есть export, pull и отдельный gex-pull; остальные операции пока предоставляются Python API.

Протокол данных

Для store используйте один JSON request с версией и вложенным наблюдением. Payload остаётся JSON-строкой, не встраивается как разобранный биржевой объект. Для load подходят NDJSON-записи с envelope version и явно завершённым потоком. При конечном процессе обязательны успешный exit status и отсутствие ошибок декодирования до использования всего результата.

Inputs может выдавать observations один раз и отдельно GEX membership по стабильным идентификаторам, назначенным в пределах ответа. Это уменьшает дублирование больших тел. Идентификатор ответа не подменяет persistent ID и не должен сохраняться клиентом как глобальный ключ.

При частичном stdout и ненулевом завершении расчёт не запускается на усечённом окне. Версия binary и версия wire protocol — разные значения. Неподдерживаемая версия отвергается до изменения базы.

Практикум H52–H54

H52 · A. Опишите три независимых версии: executable, protocol, schema.

H53 · B. Подготовьте письменную спецификацию одного request и ответа без реализации.

H54 · C. Проверьте обрезанный stdout и лишний лог в export. Приёмка: потребитель отвергает результат.

19. Подключение существующего Python-бота

Узкий адаптер

Python-адаптер знает абсолютный путь binary, сериализацию, subprocess timeout, проверку exit status и преобразование результата в ожидаемые потребителями значения. Он не реализует SQL, zlib, GEX grouping или расписание повторно.

В tunnel_archive сохраняются HTTP-работа, безопасное логирование ошибок API и collector lock. Store заменяется вызовом бинарного файла; due и prune вызываются под тем же lock. Available по-прежнему фиксируется до передачи в Rust, чтобы изменение транспорта не переопределяло временную модель.

В archived_market заменяется получение CalculationInputs, а расчётные формулы остаются прежними. На первом переходном этапе можно оставить Python-reader и мигрировать export/pull: это безопасная стадия, но не завершённая замена storage.

Контроль задержек

Даже если интерфейс синхронный, нельзя бесконечно ждать ребёнка. Родитель одновременно обслуживает stdin/stdout/stderr, ограничивает размер вывода и задаёт timeout. В асинхронном runtime блокирующий вызов должен выноситься за пределы event loop.

Не передавайте большие payload через аргументы командной строки: ограничение размера argv и видимость process list делают stdin более подходящей границей.

Практикум H55–H57

H55 · A. Перечислите обязанности Python-адаптера и запрещённые дублирования правил.

H56 · B. Составьте план замены прямых импортов в collector и calculator.

H57 · C. Проверьте недоступный binary, зависший процесс и malformed response. Приёмка: ошибки не маскируются как пустая история.

20. Тестирование и доказательство совместимости

Несколько уровней доказательства

Unit-тесты проверяют модели, bucket и кодирование времени. Module-тесты используют реальную SQLite. Integration-тесты соединяют store, reader, backup и decoder. E2E запускает настоящий binary и Python-адаптер.

Основное доказательство миграции — cross-language fixtures. Python пишет raw и compact, Rust читает. Rust пишет оба режима, Python читает. Отдельно сравниваются due, cutoff, prune, GEX grouping и failure paths. Для gzip сравниваются распакованные байты; для SQLite — данные и инварианты, а не полный файл побайтно.

Матрица отказов

Обязательны failed response, legacy schema, NULL collection, неизвестный source, повреждённый BLOB, missing payload, duplicate GEX source, неверное время, занятая база, диск без места и убийство процесса. Для SQLite backup нужен реальный WAL-case.

Текущий storage E2E подменяет SSH edge процессом, который выполняет настоящую export-команду локально. Это полезно для контракта, но не проверяет реальный VPS, key agent и known_hosts. Для релиза требуется отдельный smoke test по настоящему SSH на целевую машину.

Quality gates

Существующие Python-проверки и 92% line/branch coverage storage сохраняются, пока Python-пакет участвует в системе. Rust fmt, Clippy с отказом на warnings и cargo test дополняют их. Для Rust задайте отдельный измеримый coverage gate и failure-path matrix; нельзя объявить Python coverage покрытием Rust subprocess.

Измеряйте покрытие реальных Rust-модулей подходящим инструментом, например cargo-llvm-cov, после проверки поддержки toolchain. Высокий процент не заменяет тест COMMIT без ACK или проверку причинности.

Практикум H58–H60

H58 · A. Сопоставьте текущие tests/test_archive.py, test_inputs.py и test_transfer.py с контрактами.

H59 · B. Постройте двустороннюю fixture-матрицу Python/Rust.

H60 · C. Проведите fault injection после каждой границы записи и публикации. Приёмка: сохранены ожидаемые данные и честный статус.

21. Производительность и выбор библиотек

Что измерять

Storage в основном ждёт I/O, fsync, SQLite и SSH. Rust не делает сеть быстрее автоматически. Сравнивайте полную операцию: process startup, JSON encoding, compression, SQL, COMMIT, transport и validation.

Измеряйте длительность store, окно load, peak RSS, CPU gzip, размеры snapshot и p95 времени subprocess. Используйте одинаковые данные и настройки durability. Не сравнивайте Python с fsync и Rust без него.

Кандидаты зависимостей

Задача	Кандидат	Почему подходит
CLI	clap	Проверяемый разбор аргументов
JSON	serde и serde_json	Типизированная граница протокола
SQLite	rusqlite	Синхронные операции и backup API
zlib/gzip	flate2	Совместимые форматы сжатия
SHA-256	sha2	Адресация исходных байтов
Временные файлы	tempfile	Управляемые staging-файлы
Время	time или chrono	Проверка offsets и форматирование
Ошибки	thiserror	Явные категории библиотечных ошибок

Это кандидаты, не проверенный lockfile. Версии, features, MSRV и лицензии выбираются при реализации. Для первого CLI достаточно синхронного процесса; Tokio вводится только при необходимости конкурентных запросов или постоянного сервиса.

Bundled SQLite облегчает перенос, но увеличивает ответственность за обновление встроенной библиотеки. Dynamic SQLite уменьшает bundling, но добавляет зависимость от VPS. Обе стратегии допустимы при явном выборе.

Практикум H61–H63

H61 · A. Сформулируйте измеримую цель миграции помимо скорости.

H62 · B. Сравните один store и пакет из девяти записей с учётом startup. Приёмка: пакет не меняет незаметно транзакционную семантику.

H63 · C. Определите порог, после которого постоянный дочерний процесс оправдан измерениями.

22. Сборка и установка на VPS

Один файл не всегда полностью статический

Standalone означает независимый процесс, а не гарантированное отсутствие динамических библиотек. Linux binary должен соответствовать архитектуре VPS и доступной libc. Нативный macOS binary на Linux не запустится.

Для воспроизводимости фиксируются Rust toolchain, Cargo.lock, target triple, features, версия исходников и профиль release. Сборку проводят на совместимом Linux runner или настроенной cross-compilation среде. Статический musl-вариант выбирают после проверки зависимостей и реального запуска.

На VPS для готового binary Rust toolchain не требуется. System ssh остаётся внешней runtime-зависимостью downloader. Если выбран dynamic SQLite, он также должен быть установлен.

Установка рядом с ботом

Храните binary в versioned release-каталоге, данные — отдельно, например в существующем каталоге server_bot. Конкретные пути согласуются с текущим deployment. Путь к binary в Python-конфигурации абсолютный; планировщик не полагается на интерактивный PATH.

До переключения проверьте version, доступ к файлам от сервисного пользователя, backup реального архива и пробный read. Затем переключите ссылку на release и перезапустите нужный процесс. Замена executable и смена схемы базы — отдельные действия.

Практикум H64–H66

H64 · A. Запишите OS, CPU architecture, libc и runtime-зависимости VPS.

H65 · B. Подготовьте release manifest с checksum и protocol compatibility.

H66 · C. Запустите binary в чистом окружении сервисного пользователя без cargo и интерактивного shell profile.

23. Эксплуатация, диагностика и восстановление

Что наблюдать

Логи должны показывать операцию, длительность, asset/source при необходимости, категорию ошибки и идентификатор запроса адаптера. Не печатайте целые биржевые тела и credentials. Payload уже хранится как свидетельство и доступен отдельным диагностическим чтением.

Практические показатели: возраст последней успешной записи каждого source, число ошибок, SQLite busy, продолжительность backup, свободное место, последнее успешное скачивание и накопленный размер downloads. Размер файла после prune не является достаточной метрикой успешного удаления.

Recovery

После аварии сначала установите, цел ли исходный архив и нет ли незавершённого процесса. Затем выполните структурную validation и, при необходимости, глубокую проверку payload. Не запускайте автоматическое удаление подозрительных строк ради прохождения проверки.

Staging можно убирать после установления, что он не принадлежит активной задаче. Старые dated downloads не удаляются по умолчанию. Для них нужна самостоятельная политика хранения, не совпадающая автоматически с rolling retention VPS.

Простой клиента в пределах 24 часов не гарантирует полноту: важны реальные интервалы снимков и успешность сбора. Перекрывающиеся снимки теперь объединяются с дедупликацией. Но merge не восстанавливает отсутствующие исходные ответы и не гарантирует полноту вне наблюдаемых границ.

Практикум H67–H69

H67 · A. Отличите здоровую базу от достаточной истории для индикатора.

H68 · B. Подготовьте runbook для timeout, full disk и corrupted payload.

H69 · C. Восстановите тестовую систему из backup без доступа к биржам. Приёмка: расчёт использует только подтверждённый архив.

24. Миграция, rollback и итоговая приёмка

Переходим через законченные стадии

Сначала Rust читает копии существующих баз. Затем реализует validate и export, после этого pull и dated downloads. На этом этапе HTTP и запись остаются Python: это полезный самостоятельный выпуск transport-инструмента.

Следующий этап добавляет store, due, prune и inputs с двусторонними тестами. Python-адаптер сначала проверяется на fixtures, затем в ограниченном production-окружении. Для shadow-проверки чтения можно использовать копию архива; запуск второго живого writer ради сравнения недопустим без отдельного плана.

Полная миграция storage завершена, когда все потребители используют Rust-правила, remote export не запускает uv/Python, а Python-адаптер содержит только процессный транспорт и преобразование границы. Перенос HTTP collector остаётся отдельной задачей.

Маршрут M01–M10

M01. Зафиксировать исходные fixtures и перечень потребителей. Готовность: известны raw, compact, legacy и failure cases.

M02. Утвердить версии CLI/protocol/schema и список осознанных изменений. Готовность: timestamp formatting, tie-breaker и corruption policy описаны.

M03. Создать доменные типы и decoder. Готовность: неверный ввод не изменяет базу.

M04. Реализовать read и inputs. Готовность: cutoff и GEX совпадают с согласованной спецификацией.

M05. Реализовать backup, validate и export. Готовность: пройден реальный WAL-case и бинарный stdout чист.

M06. Реализовать SSH pull и dated download. Готовность: отказы не уничтожают прежний snapshot; gzip failure оставляет SQLite.

M07. Реализовать store, due и prune. Готовность: двусторонние Python/Rust fixtures и проверка транзакций проходят.

M08. Подключить Python-адаптер и сохранить collector lock. Готовность: нет дублирования SQL и GEX-правил в adapter.

M09. Собрать release и выполнить smoke test на том же VPS. Готовность: запуск от сервисного пользователя, корректный SSH export и измеренные расходы.

M10. Переключить потребителей и проверить rollback. Готовность: можно вернуть предыдущий binary/adapter с существующей базой либо восстановить согласованный backup.

Что проверяет rollback

Совместимая схема позволяет вернуть Python storage, но только если новые записи читаются старым кодом. Это доказывает обратная fixture-матрица. Новая проверка codec или digest может отвергать старое повреждение; возврат к менее строгому reader не исправляет данные.

Если были несовместимые migrations, возврат executable недостаточен. Нужен backup до миграции и заранее принятый способ обработки записей, появившихся после неё. Поэтому первая версия должна сохранять формат и избегать необязательных schema changes.

Практикум H70–H72

H70 · A. Разделите «Rust downloader готов» и «storage полностью перенесён».

H71 · B. Подготовьте акт приёмки M01–M10 со ссылкой на каждый test artifact.

H72 · C. Проведите тренировку rollback на копии production-архива. Приёмка: восстановлены записи и доступ потребителя; потеря данных не скрыта.

Приложение A. Матрица контрактов

Контракт	Наблюдаемое поведение	Проверка
Exact payload	UTF-8 текст не пересериализуется	Unicode и различающиеся пробелы
Store confirmation	Успех после COMMIT	Отказ записи и COMMIT без ACK
Raw/compact	Обе схемы читаются	Cross-language fixtures
Cutoff	Available в включительном окне	Boundary ±1 микросекунда
GEX	Три source одной collection	Неполная и duplicate collection
Due	Успех закрывает bucket, ошибка нет	Restart и failed request
Lock ownership	Один owner полного collector cycle	Конкурирующие collectors
Prune	Строго раньше cutoff	Общий payload старой/новой строки
Backup	Включает committed WAL	Реальный online backup
Pull	Проверка до замены	Transfer/validation failure
Gzip	Сохраняет SQLite при ошибке	Compression failure
Export	Только SQLite bytes в stdout	Настоящий subprocess
Release	Работает без Rust/Python для Rust tool	Чистое runtime-окружение

Приложение B. Словарь

Observation — сохранённый ответ с его источником, временами и collection.

Payload — точный текст ответа, не расчётный объект и не набор извлечённых полей.

Collection — идентичность группы запросов; близкое время не создаёт общей collection.

Cutoff — последняя допустимая граница available для исторического расчёта.

Bucket — UTC-интервал расписания, используемый due.

Commit — успешная фиксация транзакции SQLite; степень устойчивости к power loss зависит от согласованных настроек и среды.

Owned value — значение, сроком жизни которого управляет текущий владелец Rust.

Borrow — временный доступ к значению без передачи владения.

NDJSON — отдельный JSON-документ в каждой строке потока.

Atomic rename — переключение имени файла без наблюдаемой частичной копии на поддерживаемой файловой системе.

Idempotency — повтор операции не создаёт нового эффекта; текущий store этим свойством автоматически не обладает.

Differential test — сравнение двух реализаций на одинаковых входах с явно указанными допустимыми различиями.

Приложение C. Источники и границы проверенности

Книга основана на чтении `paper/server_bot/storage/README.md`, `pyproject.toml`, всех производственных модулей market_storage, перечня storage-тестов и integration points в tunnel_archive.py и backtest/gui_calculator/archived_market.py. Формат обучения взят из русских rust_production_book_ru, rust_tunnel_book_ru и rust_complete_book_ru этого репозитория.

Историческая проверка издания 1.0: отдельно выполнены 41 storage-тест: все прошли; combined line/branch coverage, показанное отчётом, — 94%. Проверки Black, Ruff и BasedPyright из корня также прошли. Полный корневой запуск pytest завершился успешно: 5 workspace-проверок, включая запуск дочерних project suites, за 224,22 секунды. Эти результаты относятся к существующему Python-коду, а не к будущему Rust-проекту. Rust-проект не создан, перечисленные библиотеки не собраны, их версии не зафиксированы. Реальный VPS, его architecture, libc, SSH и service configuration не проверялись. Нет измерений ускорения или памяти.

При самостоятельной реализации используйте официальные руководства The Rust Programming Language, Cargo Book, документацию SQLite Online Backup API, страницы rusqlite, serde, flate2 и системную документацию OpenSSH/systemd. Актуальные версии и features проверяются на момент сборки. Название библиотеки в книге не является обещанием совместимости любой будущей версии.

Документ разделяет фактическое поведение Python и целевые усиления: проверку digest/codec, directory fsync, явный tie-breaker, versioned protocol и обработку неоднозначного COMMIT. Каждое усиление требует собственных тестов. Подготовка этого учебника не выполняет миграцию приложения.

Дополнение к изданию 1.1: проверенная история и отложенное удаление

Предметный контракт Python и будущего Rust

Локальный файл — накопительная история. Отсутствие старого наблюдения в новом снимке VPS никогда не означает команду удаления локальной записи. Импорт сначала проверяет обе стороны, затем создаёт их объединение и лишь после этого публикует новый файл. Ошибка на любом этапе до публикации сохраняет прежний destination. Существующие датированные снимки можно импортировать командой merge по одному.

Ключ наблюдения: актив, источник, requested_at, available_at и error. Времена сначала проверяются на наличие смещения и приводятся к UTC. Одинаковый ключ и одинаковое точное тело ответа дают один результат. Collection ID и minute первой записи сохраняются; другой collection ID у точного повтора не создаёт копию. Разное тело при том же ключе — конфликт, который отменяет весь импорт. Разные available_at сохраняются: это могут быть реальные последовательные ревизии. Нельзя использовать только payload digest: неизменившаяся цена в следующий момент времени остаётся самостоятельным наблюдением.

Проверка включает структуру SQLite, обязательные поля, типы метаданных, UTC, причинный порядок requested/available, непустую коллекцию, допустимые активы и источники, codec zlib, декодирование UTF-8 и SHA256 распакованного тела. Успешное тело должно быть JSON. Это не проверка достоверности биржи и не полная валидация полей конкретного API: последняя остаётся на границах collector и calculator. Неизвестная схема, повреждённый payload и конфликт отклоняются, а не исправляются эвристикой. Ошибочные ответы сохраняются как свидетельства отказа.

Timeline без выдуманных значений

Для каждого актива диапазон ограничен первым и последним имеющимся requested_at, включая ошибочные запросы. Для каждого из девяти источников отчёт сообщает число успешных UTC-бакетов и число отсутствующих. Каденс — 300 либо 3600 секунд. Источник, отсутствующий полностью, тоже имеет пропуски внутри этого диапазона. Сбой источника не превращается в нулевой GEX. Повторы payload между разными временами не удаляются. Интерполяция и перенос предыдущей цены не выполняются.

Граница знания важнее красивой непрерывной линии: до первого и после последнего наблюдения полнота неизвестна. Пустой архив не позволяет угадать начало сбора. Несколько запросов внутри одного бакета не доказывают отсутствие внутрибакетного простоя. Пропуск не является повреждением файла; корректные оставшиеся данные сохраняются, а отчёт показывает ограничение. Команда check выполняет тот же аудит без изменения исходной базы. Это проверка наблюдаемого timeline, не обещание полной рыночной истории.

ACK после сохранения и очередь на 48 часов

Порядок протокола: согласованный export VPS, локальный download, полная проверка, merge под блокировкой, COMMIT, fsync файла, rename, fsync каталога, затем отдельный SSH acknowledge. Манифест содержит только отпечатки скачанного снимка, а не всю локальную историю и не условие «удалить всё до времени T». В отпечаток входят актив, источник, оба UTC-времени, error и точное тело ответа. ID строки и режим raw/compact не влияют на отпечаток. Конкретное кодирование Python fingerprint использует UTF-8 от JSON-массива с ensure_ascii и стандартными разделителями; Rust должен сверять Unicode, пробелы, escape и представление времени на общих fixtures, а не предполагать совпадение serde_json по умолчанию.

VPS сопоставляет отпечатки с текущими проверенными записями. Только совпавшие помечаются скопированными в copy_receipts. Поля таблицы: fingerprint как primary key, copied_at и delete_after. Время назначает VPS в UTC: delete_after равно первому copied_at плюс 48 часов. INSERT OR IGNORE сохраняет первоначальный срок при повторном скачивании. Изменённая запись и новая запись после export не попадают под старое подтверждение. Для физически повторённых одинаковых строк один отпечаток разрешает удалить все их копии после установленного срока.

Это двухэтапный восстанавливаемый протокол. Если локальная запись сохранена, но ACK не прошёл, клиент возвращает ошибку и сохраняет локальные данные. Повтор безопасен: merge не размножает строки, а ACK не переносит первый deadline. Нельзя откатывать уже опубликованный локальный файл из-за сбоя SSH. При сбое скачивания или проверки ACK вообще не отправляется. Gzip — производная копия; её сбой не отменяет сохранённый SQLite и не отзывает квитанцию.

Создание copy_receipts переводит prune в copied-only режим. Старое правило удаления по возрасту 24 часа больше не действует: нескопированные строки сохраняются независимо от возраста. После deadline collector удаляет только подтверждённые записи при следующем prune и освобождает только неиспользуемые payload. Tombstones квитанций сохраняются для идемпотентных повторов. Срок 48 часов — граница допустимости удаления; фактическое удаление зависит от следующего цикла, обычно до пяти минут позже. Остановленный сервис не гарантирует удаление точно по часам. Без регулярных скачиваний база растёт: нужен мониторинг диска, а не скрытое удаление нескопированных данных.

Порядок развёртывания

Обновите storage на обеих машинах. На VPS запустите export с copy-managed до первого копирования и перезапустите collector, чтобы он загрузил новый prune. CLI и точные пути приведены в README этого пакета. Старый collector, оставшийся в памяти или возвращённый при rollback, может игнорировать квитанции; такой rollback недопустим без отдельного плана защиты данных. Записи, уже удалённые старой retention-политикой, новый протокол восстановить не может.

Локальный cumulative archive предназначен для read-only расчётов. Его нельзя одновременно использовать как активно записываемую WAL-базу. Все импортёры должны соблюдать отдельную блокировку публикации. Проверенный набор данных не означает математическую правильность GEX: это ответственность следующего слоя.

Задачи для самостоятельного Rust-модуля

R01. Спроектируйте отдельные типы ObservationIdentity, PayloadDigest, CopyReceipt, MergeReport и SourceTimeline. Не передавайте свободные строки режимов в ядро.

R02. Реализуйте чтение raw/compact, UTC-нормализацию и проверку payload. Приёмка: битый zlib, неверный digest, неизвестный codec, naive timestamp и неверный asset отклоняются до публикации. Сравните результаты с Python на одних SQLite fixtures.

R03. Реализуйте union с транзакцией и атомарной публикацией. Приёмка: повторный импорт не добавляет строк, конфликт сохраняет предыдущие байты, два одновременных импорта не теряют данные друг друга, история вне окна VPS остаётся локально.

R04. Реализуйте timeline по каденсу источника. Приёмка: дыра 12:05 между 12:00 и 12:10 видна в отчёте; failed request не закрывает бакет; часовой источник не оценивается по пятиминутному расписанию. Пустые интервалы не заполняются.

R05. Реализуйте совместимый SSH ACK. Приёмка: байтовые fingerprint fixtures совпадают с Python, в том числе для Unicode и разных UTC offsets; запись после export и изменённый payload не помечаются старым манифестом.

R06. Реализуйте copied-only cleanup. За микросекунду до 48 часов удалений нет; на границе копии удаляются, нескопированная строка остаётся. Повторный ACK не изменяет deadline. Общий payload жив, пока на него ссылается другая строка.

R07. Проверьте crash между rename и ACK, обрыв SSH и полный диск. Клиент честно сообщает частичный исход; ни один сценарий не разрешает удалять незафиксированную локально запись. Учитывайте fsync файла и каталога отдельно от atomic visibility.

Границы проверенности издания 1.1

Rust-код не создан и не тестировался: раздел задаёт контракт учебной реализации. Рабочие изменения выполнены в Python. Отдельная storage suite на этом этапе: 59 тестов прошли, combined line/branch coverage 94,61%. Из корня workspace успешны Black, Ruff и BasedPyright (0 errors, 0 warnings), а также полный pytest: 5 workspace-тестов, включая дочерние storage, calculator и server suites, за 226,01 секунды. Изменённая storage suite дополнительно проверена после последнего усиления проверки collection minute. Browser-отчёт calculator: 98,64% V8 executed-character coverage всего chart.js; это не branch coverage. Исторические результаты издания 1.0 выше не являются новой проверкой. Реальный VPS и его действующий override в этой задаче не менялись. Автоматические проверки не доказывают отсутствие всех возможных ошибок сборщика или биржи.